POS Next — Offline Mode Plan (Summary)

Goal: a cashier can run a **complete day offline** — open a shift, clock attendance, sell, record cash movements, create customers, and close the shift — with everything syncing safely once connectivity returns.

1 · Current State

Day step	Offline status	File / location
Open the app	**Works**	`vite.config.js` (PWA cache)
Re-attach to an already-open shift	**Works**	`useShift.js` (localStorage)
Open a new shift	**Broken**	`shifts.create_opening_shift`
Clock employee attendance	**Broken**	`EmployeeAttendance.vue`
Sell all day (invoices)	**Works**	`invoice_queue` + `sync.js`
Create a customer mid-sale	**Broken**	`CreateCustomerDialog.vue`
Record a cash movement / expense	**Broken**	`DailyPaymentForm.vue`
Branch balance / payment history	**Partial**	`DailyPaymentManagement.vue`
Close the shift / Z-report	**Broken**	`ShiftClosingDialog.vue`

Root cause: the proven offline pattern (invoice queue → sync → server-side dedup) was built for invoices only, and was never generalized to the other write operations — shifts, attendance, daily payments, customers.

2 · Architecture

Instead of building four separate queues, build **one generic operation queue** and **one sync engine** that reuse the proven invoice pattern, with **server-side idempotency** via a generic tracking doctype modeled on the one already used for invoices. Each feature only registers an operation *type* — no reinventing the wheel.

3 · Phases

Phase	What it delivers	Blocks the offline day?
0	**Shared foundation.** Generic operation queue + sync engine + server-side dedup doctype + auto-sync wiring + a unified "Pending Sync" panel with per-item retry.	**Prerequisite**
1	**Shift lifecycle.** Open a shift offline + close a shift offline with a locally-rendered Z-report (instead of the server printout).	**Yes — starts & ends the day**
2	**Attendance + Daily Payment** (the two you flagged). Cache employees & picker lists + queue attendance and cash movements + server-side dedup.	**Yes**
3	**Offline customers + reliability.** Create customers offline with temp-name remapping onto queued invoices + fix stranded offline payments + fix queue-loss & worker init-race bugs + dead-code cleanup.	**Partly + prevents data loss**

4 · Suggested order

Recommended: **0 → 1 → 2 → 3**. Phase 0 is small but everything else builds on it. To see Attendance and Daily Payment working fastest, do **0 → 2** first and defer the shift work — the two tracks are independent once Phase 0 exists.

5 · Recent updates on `develop` — plan adjustments

Uncommitted feature/permission work touches several files in this plan. **None of it adds offline support**, so every gap above still stands — but it shifts a few priorities:

Update	Effect on the offline plan
Create-customer button is now **Administrator-only** (`InvoiceCart.vue`)	**Phase 3.1 refocused.** The dedicated dialog drops to LOW priority. The real risk is the **inline auto-create at checkout** in `PaymentDialog.vue` — reachable by any cashier, already caches the customer offline **but never syncs it**. Fix that path (queue + name remapping onto queued invoices).
Attendance now carries a **Shift Type** (`get_shift_types` + `shift` field)	Phase 2: the offline attendance queue payload must include `shift` ; cache the shift-type list (already degrades to an empty dropdown offline).
Invoice history is now **shift-scoped for cashiers** (`get_shift_invoices` , dual `activeResource`)	Phase 2.4: neither resource has an offline fallback. Feed cached `invoice_history` into whichever resource is active for the current user.

Payment-method switching (removed "Clear All")	**No offline impact** — pure client-side logic.

Decisions to confirm before coding:

- **Offline Z-report:** render the receipt locally at close time (recommended), or block closing until back online?
- **Standalone offline payments:** wire a real sync, or remove as dead code if the flow isn't supported?
- **Server-side idempotency:** one generic tracking doctype (recommended), or a bespoke guard per endpoint?
